```
    --dry-run                        Print the formatter output of your     ↩
                                     suite without running any examples or  ↩
                                     hooks
-w, --warnings                       Enable ruby warnings
```

We'll get to formatters in a moment. For now, let's take a closer look at three of the other options in this section:

--backtrace

RSpec normally tries to keep error backtraces short; it excludes lines from RSpec itself and from any gems you've configured. When you need more context for debugging, you can pass `--backtrace` (or just `-b`) and see the entire call stack.

--dry-run

This option, combined with `--format doc`, is a useful way to quickly get documentation-like output for your project—as long as you've taken care to phrase your example and group descriptions well.

--warnings

Ruby's warning mode can point out some common mistakes, such as instance variable misspellings. Unfortunately, Ruby will print warnings for *all* running code, including gems. If you're developing an app with lots of dependencies, you'll likely get a lot of noise in the output. But if you're developing a simple library, this option can be useful.

These options will help you drill down into details when you're diagnosing a failure, without cluttering up the output of every test run.

Setting Command-Line Defaults

We've brought these command-line options together in one chapter so that you can refer to them whenever you want to do something special for a single test run. Sometimes, though, you may need custom behavior for every run.

Rather than wasting time typing the same options over and over, you can save a set of arguments as *command-line defaults*. As the term implies, RSpec will use these by default for every run—but you can still override them.

To set defaults, save your desired options in a text file at any of the following three paths:

~/.rspec

Use this file in your home directory to store *global personal preferences*. RSpec will use it for any project on your machine. For instance, you may prefer to stop your test run on the first failed example, while your